Filtered merges make deeper representation trees pay: Subset Sum and Meet-LWE

Nick Howgrave-Graham

November 2026

Abstract

Dubiner's bucketing codes, used as type-class filters on representation coordinates, let a merge avoid generating matched pairs with the wrong split pattern. We show that in representation trees their main effect is to make depth pay. For random density-one Subset Sum they improve the exponent of Bonnetain, Bricout, Schrottenloher and Shen from 0.2831 (our recomputation) to 0.2690, at depth 5; no better point was found up to depth 9. For May's Meet-LWE on ternary keys they improve the asymptotic exponent by 0.013–0.023 for key weights $\omega \geq 0.375$. Filters can pay only at merges where most matched pairs have the wrong split pattern, and do pay where such merges bind; we show both where this holds and where it fails. A survival recursion justifies the accounting, a symmetry lemma shows that the filters cannot bias the planted solution, and simulations support the remaining heuristics.

Keywords: subset sum; representation technique; nearest-neighbour search; Dubiner bucketing codes; locality-sensitive filtering; Meet-LWE; ternary LWE; cryptanalysis.

1 Introduction

An instance of Subset Sum consists of integers $a_1, \ldots, a_n$ and a target t, and asks for $x \in \{0,1\}^n$ with $\langle a, x\rangle = t$. In the *random* setting the a_i are uniform in $[0, 2^n)$ and $t = \langle a, x_0\rangle$ for a planted x_0 of weight $n/2$ (density one). We measure the exponent $\log_2(\text{time})/n$.

Meet-in-the-middle [11, 20] gives $2^{n/2}$. Howgrave-Graham and Joux [12] introduced *representations*: the solution is written in many ways as a sum of two vectors, so a random modular constraint can be imposed on the summands while some representation survives. Becker, Coron and Joux [5] allowed digits -1, and Bonnetain, Bricout, Schrottenloher and Shen [6] added the digit 2 and an accounting in which intermediate lists may be random subsets of their constrained sets. Table 1 summarises.

Dubiner's bucketing codes. Dubiner [9] introduced bucketing codes for the statistical nearest-neighbour problem: random codewords of a fixed type, into which each point is placed according to its joint type with the codeword, so that correlated pairs share a bucket far more often than independent ones. He gave the information-theoretic tradeoff that governs them, and our cost terms (Proposition 3) instantiate it. May and Ozerov's consistency filtering [19] and the locality-sensitive filters of Becker, Ducas, Gama and Laarhoven [2] and Christiani [7] are in the same line. Our filters are Dubiner's codes specialised to a representation merge, an idea whose reach in representation algorithms has not been fully exploited.

exponent	work	digit set	depth	merges
0.337	HGJ [12] (as corrected in [5])	$\{0,1\}$	3	plain
0.329	May–Ozerov [19]	$\{0,1\}$	2	consistency filtered
0.291	BCJ [5]	$\{-1,0,1\}$	3	plain
0.287	May–Ozerov [19]	$\{-1,0,1\}$	3	consistency filtered
0.2852, 0.2814	this work†	$\{-1,0,1\}$	3, 4	type-class filtered
0.2830	BBSS [6]	$\{-1,0,1,2\}$	3	plain
0.2793	this work	$\{-1,0,1,2\}$	3	type-class filtered
0.2727	this work	$\{-1,0,1,2\}$	4	type-class filtered
0.2690	this work	$\{-1,0,1,2\}$	5	type-class filtered

Table 1: Heuristic time exponents for random density-one Subset Sum. Depth is the number of levels of representation lists below the root (d in Section 2); BBSS's "five levels, numbered 4 down to 0" [6, §2.5] is $d = 3$ with meet-in-the-middle leaves; our recomputation of [6] gives 0.2831 (Table 2). †BBSS accounting (Heuristic 1); under the strict accounting of [5], which [19] also uses, our filtered depth-3 value is 0.2863, which a depth-4 search under the same accounting did not improve, and Ozerov's accounting gives HGJ 0.340 rather than 0.337, so the rows are not directly comparable.

A worked example. Take the top merge of the HGJ tree [12]. The solution $x_0 \in \{0,1\}^n$ of weight $n/2$ is written as $x + y$ with x, y of weight $n/4$ and disjoint supports, and two lists of such vectors are joined on $\langle a, \cdot \rangle$ modulo M. *Why there is waste:* a random pair of weight-$n/4$ vectors is disjoint with probability $\binom{3n/4}{n/4}/\binom{n}{n/4} = 2^{pn+o(n)}$, where $p = \frac{3}{4}h(\frac{1}{3}) - h(\frac{1}{4}) = H(\sigma) - 2H(\delta) \approx -0.1226$, so almost every modular match overlaps and is discarded. *How a filter removes it:* take a random $u \in \{L, R\}^n$ with $n/2$ labels L, let x enter u if its support lies in the L-coordinates and y if its support lies in the R-coordinates, and match only vectors that entered the same u. A disjoint pair enters a common u with probability $2^{-I(XY;\Lambda)n} = 2^{-n/2}$, a single vector with probability $\binom{n/2}{n/4}/\binom{n}{n/4} = 2^{-I(X;\Lambda)n}$, $I(X;\Lambda) = I(Y;\Lambda) \approx 0.3113$, and an overlapping pair never. The saving $I(X;\Lambda) + I(Y;\Lambda) - I(XY;\Lambda) \approx 0.1226$ is the whole waste: every pair generated is valid. The price is that each vector lies in $2^{(I(XY;\Lambda)-I(X;\Lambda))n} = 2^{0.1887n}$ filters, which here costs more than the whole algorithm ($0.3113 + 0.1887 = 0.5$). *Where it pays:* not at this merge, which does not bind; in the HGJ tree of depth 3 it examines $2^{0.1226n}$ pairs against $T = 0.3371$. The binding merges are the two below it, with smaller waste, $p_2 \approx -0.026$ and $p_3 \approx -0.006$, and there softer labellings, in which the labels are only correlated with the digits, save 0.010 and 0.006 at an acceptable entry cost, giving $0.3371 \to 0.3269$ (Table 2; `floor/hgj_example.py`). So filters can pay only where $2^{p_j n} \ll 1$, and do pay where such a merge binds; when they make the upper merges cheap, extra levels pay too.

Our approach. We look at where the work of a representation tree goes. At each merge, two lists are joined on a modular constraint, and most of the matching pairs are then discarded because their split pattern is wrong; only a fraction $2^{p_j n}$ survive. We ask what happens if these pairs are never generated, by filtering on the joint coordinate type as in the example above. Such filters can save at most this waste, so they can pay only where $2^{p_j n} \ll 1$, and they do pay where such a merge binds. We find both cases: where filters pay (Sections 5 and 7) and where they do not. On the BBSS tree at its own depth the filters gain only about 0.004 ($0.2830 \to 0.2793$). BBSS observed [6, Remark 4] that the time "is a decreasing function of the number of levels, which quickly reaches

a minimum", because "adding one more level does not change the cost of the upper levels, which will remain the most expensive"; Ozerov makes the same observation for ternary trees [19, §4.1]. Filters change exactly this. They make the upper merges cheap, so additional levels, which shrink the lists, start to pay; at the depth-4 optimum the costs of all four merges tie, and at the depth-5 optimum the top four tie while the sparse bottom merge falls below T. This gives the improvement $0.2830 \to 0.2690$ (Section 5): about 27% of it comes from the filters at fixed depth, and 73% from the depth they make affordable. In decoding the analogous effect is known: with near-neighbour search at every level, BJMM-type trees improve up to depth 4 [3, 4]. The same happens in May's Meet-LWE [17]: once the split laws are re-optimised with filters, the denser levels carry pattern waste that the filters remove, and the exponent improves by 0.013–0.023 for key weights $\omega \geq 0.375$; the gain appears at the first extra level that the filters make affordable (Section 7).

What is new. Filtering of representation trees is due to May and Ozerov [19], for the HGJ and BCJ digit sets at depth at most 3. Filters combined with depth are known in decoding, from Both and May [3, 4]. New here are filters on the BBSS digit set $\{-1, 0, 1, 2\}$, trees of depth $d \geq 4$, and the soundness analysis, and the first use of such filters in Meet-LWE, where they improve May's exponents once the split laws are re-optimised.

Contributions.

1. Type-class filtered τ-merges on representation trees with general digit sets, with cost terms in closed form and the observation that the filter law only matters through its label type (Section 3).
2. An optimisation of the BBSS tree with filters at every merge, for depths $d \leq 9$: 0.2727 at depth 4 and 0.2690 at depth 5, with nothing better found at greater depth (Section 5).
3. Soundness of the accounting: a survival recursion that is satisfied with equality by every reported optimum, and a symmetry lemma showing that the planted solution survives each filtered merge with exactly the average probability (Section 4).
4. Experiments testing survival, filter bias and sibling independence (Section 6).
5. A second application: filtered Meet-LWE trees for ternary LWE keys, improving May's Rep-1 exponents by 0.013–0.023 for $\omega \geq 0.375$ (Section 7).

What is proved. All results are heuristic, in the same sense as [12, 5, 6]. Proved are Lemma 2, Proposition 3 (given the heuristics), Remark 4, Proposition 5, the survival recursion together with the check that every reported optimum satisfies it, and Lemma 6. Assumed are Heuristics 1 and 2, which the experiments of Section 6 support. Numerical are the exponents: each is the best point the optimiser found, not a proven optimum of the model, and the limit of about 0.269 is the observation that depths 6–9 gave nothing better.

Related work. Consistency filtering for HGJ and BCJ trees is in Ozerov's thesis [19, Ch. 2, §4.2], reporting joint work with May, which leaves digits ± 2 as an open question [19, after Cor. 65]; BBSS list it as "not studied" [6, Table 1]. Böhme [1] reportedly reached 0.2871 with $\{-2, \ldots, 2\}$ (unpublished; see [6, fn. 5]). Filtering by joint type is also the mechanism of May–Ozerov near-neighbour search in decoding [18]; Christiani [7] gives the asymmetric framework for locality-sensitive filters. In the decoding trees of Both and May [3, 4] and the ternary-LWE trees of

Kirshanova and May [15] (see [14, App. A] for a correction to its analysis), near-neighbour search replaces exact matching of values by approximate matching, and the representation coordinates are checked after each merge; our filters act on the representation coordinates and keep exact modular matching. An earlier claim of $2^{0.255n}$ [10] has been withdrawn ("issue with counting duplicate representations"); unlike [10] we never sample or modify list contents, and all representation counts are those of [6]. Li [16] claims $2^{0.24n}$ using a ternary-tree representation structure; we were not able to reproduce this exponent within the heuristics used here.

2 Preliminaries

Logarithms are to base 2 and H is Shannon entropy. For a finite set Σ and $v \in \Sigma^n$ the *type* of v is its empirical distribution; there are $2^{nH(P)+O(\log n)}$ strings of type P [8]. Exponents are stated up to $O(\log n)/n$.

Digit sets, sets and merges. A *digit set* is a finite $\mathcal{D} \subset \mathbb{Z}$ containing 0 and 1; we use $\{-1, 0, 1, 2\}$ unless stated otherwise. For a modulus m, residue $\tau \in \mathbb{Z}_m$ and type w on $\mathcal{D}$, the (m, τ, w)-*set* is $\{x \in \mathcal{D}^n : \ \mathrm{type}(x) = w, \ \langle a, x\rangle \equiv \tau \pmod m\}$. A τ-*merge* of lists $L_A \subseteq$ an (m', τ_A, w')-set and $L_B \subseteq$ an $(m', \tau - \tau_A, w')$-set, with $m' \mid m$, outputs the $x + y$ ($x \in L_A$, $y \in L_B$) with $\langle a, x + y\rangle \equiv \tau \pmod m$ and $\mathrm{type}(x + y) = w$, i.e. elements of the (m, τ, w)-set. Outputs are kept as a set (no duplicates).

The tree of depth d. Level 0 is the solution x_0, of type $\delta_0 = (\frac{1}{2}, \frac{1}{2})$ on $\{0, 1\}$. Levels $j = 1, \ldots, d$ carry 2^j lists of type-δ_j vectors, constrained modulo $M_j = 2^{c_j n}$ with $1 = c_0 \geq c_1 \geq \cdots \geq c_d \geq 0$, $M_j \mid M_{j-1}$, and residues chosen at random consistently down the tree (the τ's). Each level-$(j-1)$ list is the τ-merge of two level-j lists; the level-d lists are built by meet-in-the-middle from half-vectors, in time $2^{H(\delta_d)n/2}$. The depth d is the number of levels of lists below the root.

Splits. A level-$(j-1)$ digit z is written $z = x + y$ with $x, y \in \mathcal{D}$; the *split law* σ_j is the joint type of (x, y) over all coordinates. It is symmetric, has both marginals δ_j, and is determined by δ_{j-1} and, for each parent digit, a distribution over its unordered splits (for $\{-1, 0, 1, 2\}$: three free fractions, $0 \to (\pm 1, \mp 1)$, $1 \to (2, -1)$, $2 \to (1, 1)$). Write $H_j = H(\delta_j)$. A fixed level-$(j-1)$ vector has $2^{\rho_j n}$ representations of split type σ_j, and a random pair of type-δ_j vectors is such a split with probability $2^{p_j n}$, where

$$\rho_j = H(\sigma_j) - H_{j-1}, \qquad p_j = H(\sigma_j) - 2H_j.$$

Heuristic 1 (BBSS [6])**.** *Every list behaves like a uniformly random subset, of its stated size, of its (M_j, τ, δ_j)-set. The merge of two level-j lists of size $2^{\ell_j n}$ examines $2^{P_j n}$ pairs, $P_j = 2\ell_j - (c_{j-1} - c_j)$, and outputs $2^{(P_j + p_j)n}$ vectors; the expected number of solutions found at the root is at least* 1*.*

3 Filtered τ-merges

Fix a merge of two level-j lists and drop the index. Let $(X, Y) \sim \sigma$, and let Λ take values in a finite *label set* with conditional law $\kappa(\cdot \mid x, y)$ (one law per ordered pair type). Write Q for the law of (X, Y, Λ) and Q_Λ for its label marginal.

Definition 1. A *filter* is a string u in the label set, of type Q_Λ. A left vector x *enters* u if the joint type of (x, u) is the (X, Λ)-marginal of Q, and a right vector y enters u if the joint type of

(y, u) is the (Y, Λ)-marginal. The filtered τ-merge draws $F = 2^{I(XY;\Lambda)n}$ random filters and, within each, matches the entering left and right vectors on the new constraint bits.

These are Dubiner's random bucketing codes [9], specialised to a representation merge; with a two-symbol label set and recursive block structure they are Ozerov's consistency algorithm [19, §2.2]. Figure 1 compares the two merges.

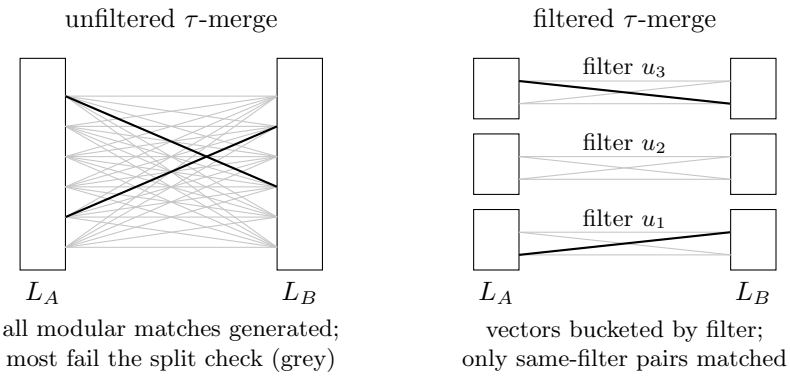


Figure 1: Schematic of a τ-merge without and with filters. Thick lines are valid splits, grey lines are matched pairs with the wrong split pattern. With filters, each vector is placed in the filters it enters and only pairs that share a filter are matched, so most of the wasted pairs are never generated.

Lemma 2. *Up to $2^{O(\log n)}$: a pair of joint type σ enters a common random filter with probability $2^{-I(XY;\Lambda)n}$; a left (right) vector enters with probability $2^{-I(X;\Lambda)n}$ ($2^{-I(Y;\Lambda)n}$); a pair of independent type-δ vectors enters a common filter with probability $2^{-(I(X;\Lambda)+I(Y;\Lambda))n}$.*

Proof. This is Dubiner's method-of-types argument [9]: for fixed (x, y) of type σ, $2^{nH(\Lambda|XY)}$ of the $2^{nH(\Lambda)}$ strings of type Q_Λ have joint type Q with it; similarly for single vectors; for independent x, y the two events are independent given u. □

Heuristic 2. *The pairs that agree on the constraint bits behave, with respect to filters, like independent uniform pairs of type-δ vectors.*

Proposition 3. *Under Heuristics 1 and 2, the filtered τ-merge outputs $2^{(P+p)n}$ vectors up to a constant factor, at cost dominated by*

$$\underbrace{\ell + I(XY;\Lambda) - I(X;\Lambda)}_{\text{entries (left)}}, \quad \underbrace{\ell + I(XY;\Lambda) - I(Y;\Lambda)}_{\text{entries (right)}}, \quad \underbrace{P + I(XY;\Lambda) - I(X;\Lambda) - I(Y;\Lambda)}_{\text{pairs passing}},$$

plus the cost of decoding, for each vector, the filters it enters.

With $F = 2^{I(XY;\Lambda)n}$ filters a valid pair shares $\Theta(1)$ filters in expectation, so a constant fraction of valid pairs could be missed; we use nF filters, which makes the miss probability $e^{-\Omega(n)}$ per pair at a polynomial cost. A filter pays when $I(X;\Lambda)+I(Y;\Lambda) > I(XY;\Lambda)$; we call $I(X;\Lambda)+I(Y;\Lambda)-I(XY;\Lambda)$ the *saving* of the merge.

Remark 4 (Only the label type matters). One may draw filters i.i.d. from any law π on the label set; the number of filters and the entry probabilities then change by factors $2^{\pm D(Q_\Lambda\|\pi)n}$ that cancel in all three cost terms, since only filters of type Q_Λ ever contain a valid pair. We therefore normalise $\pi = Q_\Lambda$ (see also Appendix A).

Decoding the filters. Each vector must find the $2^{(I(XY;\Lambda)-I(X;\Lambda))n}$ filters it enters, among $2^{I(XY;\Lambda)n}$, in time close to that number. Random codes do not allow this, so we use a product construction, as in Dubiner's bucketing codes [9] and the tensored filters of [2]; it does not need the code rates to be small.

Proposition 5. *The filtered τ-merge can be implemented so that each cost term of Proposition 3 is achieved up to a factor $2^{o(n)}$, including the time to decode the filters, while each valid pair of type σ is still found with probability $1-o(1)$.*

Proof. Apply a uniformly random permutation to the coordinates (the instance is exchangeable) and cut them into $b=\lceil\sqrt{n}\rceil$ blocks of size $s\geq\lfloor n/b\rfloor$. For a valid pair, the vector of its per-block joint types, its *pattern*, is a sequence $(\sigma_1,\dots,\sigma_b)$ averaging to σ. Call the pattern *typical* if $\|\sigma_i-\sigma\|_\infty\leq\epsilon_n=n^{-1/4}\log n$ for every i. Fix a valid pair. After the permutation, each block is a sample of s coordinates drawn without replacement from its n coordinates, so for each joint digit type t of the pair, Hoeffding's inequality for sampling without replacement [13, §6] gives

$$\Pr\bigl[|\sigma_i(t)-\sigma(t)|>\epsilon_n\bigr]\leq 2e^{-2s\epsilon_n^2}=2e^{-2(1-o(1))\log^2 n}=n^{-\Theta(\log n)}.$$

A union bound over the b blocks and the $O(1)$ joint types shows that the pattern is typical with probability $1-O(\sqrt{n})\,n^{-\Theta(\log n)}=1-o(1)$. The block types of x and of y are marginals of the pair's block types, so they are within ϵ_n of δ as well. A block type takes at most $(s+1)^{O(1)}$ values, so there are at most $(\sqrt{n}+1)^{O(b)}=2^{O(\sqrt{n}\log n)}$ typical patterns.

For each pattern we build an independent filter family: in block i, a random code C_i of $2^{\sqrt{n}\,I_{\sigma_i}(XY;\Lambda)}$ label strings of type $Q_{\Lambda,i}$ (the label marginal of $\sigma_i\times\kappa$), and a filter is a tuple $(u_1,\dots,u_b)\in C_1\times\cdots\times C_b$. A left vector enters the tuple if it enters every u_i in the sense of Section 3, with block-i marginals; similarly on the right. A vector determines only its own sequence of block types, not the joint pattern of a pair, so it enters the filters of every family whose block marginals on its side equal its own block types, and of no other family. In particular, a vector with atypical block types enters no family at all, since families are built only for typical joint patterns. There are at most $2^{O(\sqrt{n}\log n)}=2^{o(n)}$ families, so the entry and decoding costs of a vector, summed over all families compatible with it, exceed the single-family cost by at most a factor $2^{o(n)}$; passing pairs are counted per family, with the same factor. A valid pair has a typical joint pattern with probability $1-o(1)$ by the bound above, and is then found in the family of its pattern. A vector decodes its filters by testing, in each block, all of C_i (time $2^{O(\sqrt{n})}$) and then enumerating the product of the per-block lists, in time proportional to the output plus $b\,2^{O(\sqrt{n})}$.

Lemma 2 applies in each block. The probabilities multiply, so a valid pair with this pattern shares $\Theta(1)$ filters in expectation (amplified as above), and the entry and passing exponents are block averages of the single-block terms. Those terms are continuous functions of the pair type for fixed κ, so each average differs from the global term of Proposition 3 by $O(\epsilon_n)$, i.e. by $o(n)$ in the exponent. Summing over the $2^{o(n)}$ patterns multiplies all costs by $2^{o(n)}$. □

4 Accounting and soundness

The program. For fixed split laws and filter laws, the exponents (c_j,ℓ_j,T) are the variables of a linear program: minimise T subject to Heuristic 1 ($\ell_{j-1}=P_j+p_j$, $\ell_j+c_j\leq H_j$, root yield ≥ 1), nesting $c_j\leq c_{j-1}$, and T at least the leaf build $H_d/2$, every list size, and, at every merge, the three terms of Proposition 3 (or P_j if unfiltered). The split and filter laws are optimised by repeated Powell and Nelder–Mead searches, warm-started from shallower optima, from deeper ones, and from optima for wider digit sets with the extra digits projected out; each reported value is the best

over all these starts. The depth is fixed, with per-level constants; trees whose depth grows with n are outside the model.

Survival recursion. Heuristic 1 allows $c_j > \rho_j$ at a level (fewer representations than the constraint removes), which is not obviously sound: a level-$(j-1)$ element then survives only with probability $2^{-(c_j-\rho_j)n}$. Let $s_j \le 0$ be the log-survival exponent of a level-j element, with $s_d = 0$ (complete leaves). Saturating the Poisson survival law $\sigma = 1 - e^{-c\sigma_A\sigma_B}$ gives

$$s_j = \min\bigl(0,\ \rho_{j+1} - c_{j+1} + 2s_{j+1}\bigr) \quad (j = d-1, \dots, 1), \qquad \rho_1 - c_1 + 2s_1 \ge 0 \text{ (root)},$$

with list sizes $\ell_j = H_j - c_j + s_j$. Excess representations at a split compensate deficits deeper in the tree (closer to the leaves); since survival is capped at 1, they cannot be carried upward to pay for a deficit closer to the root. So the accounting of Heuristic 1 is sound exactly when every deficit lies below a sufficient excess. We solve the program with these constraints in place of Heuristic 1 (exactly, by enumerating which levels saturate). **Every reported optimum gives the same value under both accountings.** At depth 4, only the split into level 2 has a deficit ($s_1 = -0.071$), paid exactly by the root excess $\rho_1 - c_1 = 0.141 = -2s_1$ (Table 3).

Symmetry. Filtered lists are not uniform random subsets in any obvious sense, but the planted solution is not disadvantaged.

Lemma 6. *Fix a node (per node), its exact type, and the randomness: the instance a with i.i.d. uniform coordinates, the node's residue τ, marginally uniform and independent of a, the residues below it (functions of τ and fresh independent randomness), and i.i.d. filters independent of all of these. Let O be the output list and V_τ the parents of the exact type in the node's residue class. Then for every parent P of that type,*

$$\Pr[P \in O \mid P \in V_\tau] = \mathbb{E}|O| \,/\, \mathbb{E}|V_\tau|,$$

and the same holds at every level.

Proof. Every step (complete coset leaves, modular joins, validity checks, the exact-type output restriction, and filter entry, which depends only on joint (digit, label) counts) is equivariant under coordinate permutations g: $O(ga, \tau, gu) = g\,O(a, \tau, u)$. The law of (a, τ, u) is g-invariant and g maps V_τ to itself, so $\Pr[P \in O \wedge P \in V_\tau]$ is the same for all P of the type, on which the permutations act transitively. Summing over P and dividing by $\Pr[P \in V_\tau] = 1/M$ gives the claim; child lists are outputs of equivariant steps, so induction over levels applies.

The hypothesis on residues holds in the planted model. The root target $t = \langle a, x_0\rangle$ depends on a, but at every node the residue is τ or $t - \tau$ (reduced modulo the node's modulus) for a τ that is uniform and independent of (a, x_0), so each node residue is marginally uniform and independent of a. The planted parent P^* at a node is a function of x_0 and of the split choices, so it is independent of a, and the lemma applies to it. □

The lemma concerns the ratio of expectations, which is the quantity the accounting uses; it says nothing about joint survival of two nodes. In particular sibling residues τ and $t - \tau$ are dependent through t; this joint dependence is what the sibling-independence experiment of Section 6 tests.

digit set	merges	$d=2$	3	4	5	6	7	8
$\{0,1\}$	plain	0.3371	0.3371	0.3371	0.3371	0.3371	0.3371	0.3371
	filtered	0.3269	0.3269	0.3269	0.3269	0.3269	0.3269	0.3269
$\{-1,0,1\}$	plain	0.3122	0.2891	0.2891	0.2891	0.2891	0.2891	0.2891
	filtered	0.3122	0.2852	0.2814	0.2814	0.2814	0.2814	0.2814
$\{-1,0,1,2\}$	plain	0.3025	0.2831	0.2831	0.2831	0.2831	0.2831	0.2831
	filtered	0.3025	0.2793	0.2727	0.2690	0.2690	0.2690	0.2690

Table 2: Time exponents by depth d, BBSS accounting, plain and filtered merges (each entry is the best found; a deeper tree can copy a shallower one exactly, by a bottom level of trivial splits $z \to (z,0)$ with an unfiltered merge, so entries are monotone). $\{0,1\}$ and $\{-1,0,1\}$ under BBSS accounting give 0.3371 and 0.2891; the strict accounting of [12, 5] gives 0.3371 and 0.2909.

5 Results

Theorem 7 (heuristic)**.** *Under Heuristics 1 and 2, random density-one Subset Sum can be solved in time* $2^{(0.2690+o(1))n}$ *with a filtered* $\{-1,0,1,2\}$ *tree of depth* 5*, and in time* $2^{(0.2727+o(1))n}$ *with depth* 4*. The* $o(1)$ *comes from filter decoding (Proposition 5).*

Structure of the optimum. Table 3 gives the depth-4 point. The filtered passing costs of all four merges tie at T. At this point the filters save 0.089 and 0.044 at the dense levels 1–2, only 0.011 at level 3, and nothing at the last merge. The deficit at level 2 is paid by the root excess, and the survival recursion is tight. The depth-5 optimum (Table 6) keeps this shape one level further down: the filters save 0.088, 0.071 and 0.068 at the top three merges, 0.009 at the fourth and nothing at the last, and the bottom level is sparse ($\delta_5(0) = 0.92$) with almost no 2s. The extra level pays because it lets the third merge, where pattern waste is still large, be filtered at smaller lists. From depth 6 on the best points are copies of the depth-5 point (Table 2); independent searches at depths 6–9 came within 0.0001 of it but not below. Lemma 6 holds at every level, sparse ones included, so these merges need no separate bias argument (see also Section 6).

Where filters pay. The column $p_j = H(\sigma_j) - 2H_j$ of the parameter tables is the log-probability that a random matched pair is a valid split, and a filter can only save what this pattern waste costs. At the depth-5 point p_j is -0.36, -0.11 and -0.13 at the top three merges, and -0.05 and -0.006 at the two bottom ones, and the savings follow the same profile. At the dense top levels a random pair has overlapping supports, and the cancellations $0 \to (1,-1)$ and $1 \to (2,-1)$ need matching digits on the same coordinate, which random pairs rarely have. At the sparse bottom levels supports rarely meet and the splits are mostly $1 \to (1,0)$, so a random matched pair is almost always valid and there is nothing to filter. (At earlier, slightly worse depth-6 and 7 optima, a sparse level dominated by $1 \to (2,-1)$ splits carried waste $p_j \approx -0.18$ and was filtered as well.)

Where filters do not pay. The same test predicts the failures. At the bottom levels above, $p_j \approx 0$ and the filters are nearly inactive. In the sparse-key trees of Hhan, Kim, Lee and Son [14] the binding merges split without cancellations (each child carries half of the parent's weight), and a random matched pair is then almost always valid (two random weight-10 vectors in dimension about 2000 have disjoint supports with probability about $2^{-0.04}$); the filters save about 3.7 bits only at the top merge, which has cancellations but does not bind, and gain nothing. In Meet-LWE

at May's own parameters the binding merges have $p_j \geq -0.06$, and the filters gain nothing until the split laws are re-optimised (Section 7).

Memory. The memory exponent, the largest of the leaf lists, the stored lists and the filter entries $\ell_j + I(XY;\Lambda) - \min(I(X;\Lambda), I(Y;\Lambda))$, is 0.2727 at depth 4 and 0.2690 at depth 5 (last line of each parameter table). At every reported point it equals or nearly equals the time, because the entry terms tie with the passing terms at the optimum. The gain over BBSS, 0.0142 (from our recomputed 0.2831), splits into 0.0038 from filters at depth 3 and 0.0103 from the extra depth.

j	$\delta_j(-1)$	$\delta_j(0)$	$\delta_j(1)$	$\delta_j(2)$	ρ_j	c_j	s_j	ℓ_j
1	0.0534	0.6497	0.2905	0.0064	1.0285	0.8872	-0.0707	0.2369
2	0.0573	0.7614	0.1803	0.0010	0.7088	0.7794	0.0000	0.2124
3	0.0523	0.8335	0.1136	0.0006	0.5449	0.5449	0.0000	0.2593
4	0.0324	0.9040	0.0636	0.0000	0.2722	0.2722	0.0000	0.2727

merge of level j	pairs	p_j	$I(XY;\Lambda)$	entries gap	saving	passing
1	0.3611	-0.3611	0.1601	0.0357	-0.0886	0.2725
2	0.3171	-0.0801	0.1650	0.0603	-0.0443	0.2727
3	0.2841	-0.0717	0.0382	0.0134	-0.0114	0.2727
4	0.2727	-0.0134	0.0000	0.0000	0.0000	0.2727

leaves $H_4/2 = 0.2725$; root excess $\rho_1 - c_1 = +0.1413$, $2s_1 = -0.1413$; memory 0.2727; $T = 0.27272$

Table 3: The depth-4 optimum: levels (top) and merges (bottom). Columns: $\delta_j(\cdot)$, the digit type of level j; ρ_j, the representation exponent of a level-$(j-1)$ vector into level-j pairs; c_j, the constraint bits ($M_j = 2^{c_j n}$); s_j, the survival exponent of Section 4; ℓ_j, the list size; *pairs*, $P_j = 2\ell_j - (c_{j-1} - c_j)$, the pairs an unfiltered merge would examine; $p_j = H(\sigma_j) - 2H_j$, the log-probability that a matched pair is a valid split; $I(XY;\Lambda)$, the code rate (at $\pi = Q_\Lambda$); *entries gap*, $I(XY;\Lambda) - \min(I(X;\Lambda), I(Y;\Lambda))$, the log number of filters per vector; *saving*, $I(XY;\Lambda) - I(X;\Lambda) - I(Y;\Lambda)$ (negative means fewer pairs); *passing*, pairs + saving, the pairs the filtered merge examines. The last line gives the leaf build, the root condition and the memory exponent. All exponents are per n.

Saturation. The exponent saturates with depth: 0.2793, 0.2727, 0.2690 for $d = 3, 4, 5$, and 0.2690 again for $d = 6, \ldots, 9$ (Table 2). Each is a computed point, the best found over all starts, not a proven optimum of the model, so 0.2690 is not a lower bound for it. At depths 6–9 we searched from the depth-5 point extended by one level (trivially, by grafting a copy of a level, and with a new sparse bottom level), from the earlier deeper optima and from random perturbations, for several hundred CPU hours; the best independent points reached 0.26896–0.26906, and none went below the depth-5 value. This is what the structure of the point leads one to expect. At depth 4 the bottom merge and the leaves bind (0.2727 and 0.2725), so a fifth level pays; at depth 5 they are slack (0.2670 and 0.2372 against $T = 0.2690$), and T is set by the top four merges, whose costs depend only on levels 1–4. A further level can only make the slack bottom cheaper, which is the saturation of [6, Remark 4], moved from depth 3 to depth 5 by the filters. A deeper tree with different upper levels is not excluded by this argument, but our searches found none. We therefore claim only that the exponent converges to about 0.269. Deeper trees alone do not lead to subexponential time within this accounting.

Wider digit sets. Once both digit sets are optimised from the same starting points, adding the digit -2 gains little: in such matched comparisons at earlier optima the gain was nothing at depth 4, 0.0003 at depth 5 and 0.0001 at depth 6 (we did not repeat them at the final points). The digit 3 has zero marginal value, since it can only arise from splitting a 2, and an optimisation over $\{-4, \ldots, 4\}$ did not converge and was stopped. We therefore keep the BBSS digit set $\{-1, 0, 1, 2\}$ throughout.

Label sets. Re-optimising the filters at an earlier depth-5 point with a 4-symbol label set, warm-started from the 3-symbol optimum, gives 0.273224 against 0.273335: a gain of 0.0001, so 3-symbol filters appear essentially optimal.

6 Experimental validation

Survival. Table 4: a depth-3 HGJ tree (weights $n/2 \to n/4 \to n/8 \to n/16$) with complete leaves (leaf ratio $c \approx 3.9$) and internal ratio $c \in \{1, 2, 3, 4\}$. The Poisson model $\sigma = 1 - e^{-c\sigma_A\sigma_B}$ is slightly optimistic at every level, but survival at $c = 3$ does not decay, and the work per success is flat in c to within about one bit (conservation: raising c buys nothing).

n (draws)	c	σ_2	Poisson	σ_1	Poisson	Pr[x_0 found] (Poisson)	$\log_2$ work/success
48 (40)	1	0.953	0.982	0.593	0.618	0.15 (0.32)	21.3
	2	0.954	0.982	0.823	0.856	0.65 (0.77)	20.5
	3	0.943	0.982	0.906	0.943	0.98 (0.93)	20.8
	4	0.957	0.982	0.957	0.979	1.00 (0.98)	21.4
64 (10)	1	0.963	0.980	0.602	0.617	0.30 (0.32)	25.3
	2	0.967	0.980	0.841	0.853	0.50 (0.77)	26.0
	3	0.953	0.980	0.928	0.944	0.90 (0.93)	26.1
	4	0.964	0.980	0.972	0.978	1.00 (0.98)	26.6

Table 4: Survival by level in a real HGJ tree (`floor/survival.py`). $\sigma_j = |L_j|/\mathbb{E}|V_j|$. At $n = 64$ the modulus is capped at 2^{56} (density 1.14).

Filter bias. A filtered merge at $n = 32$ shaped like the level-2 merge of an earlier depth-4 optimum (complete coset lists of about 1900 elements, a 3-symbol label set from the optimum's conditionals, 300 draws): the planted parent survives with probability 0.8633 against mean $\sigma = 0.8638$ without filters, and 0.7200 ± 0.0259 against 0.7150 with filters, as Lemma 6 predicts. A second merge at $n = 20$ is shaped like the sparse bottom merge of an earlier depth-6 optimum: the parent has four 1s and two -1s, and the children have two 2s and three -1s, joined by $1 \to (2, -1)$ and $-1 \to (-1, 0)$ (`floor/bias_sim_sparse.py`, 400 draws). The planted parent survives with probability 0.598 ± 0.025 against $\sigma = 0.611$ without filters, and 0.438 ± 0.025 against 0.445 with filters.

Sibling independence. Two planted siblings (disjoint supports, shared instance, independent residues), each built by its own filtered merge, 4000 draws per n: $\Delta = \log_2 \Pr[\text{both}] - \log_2 \Pr[A] - \log_2 \Pr[B]$ is $+0.0005$, $+0.0091$, -0.0048, $+0.0077$ at $n = 24, 32, 40, 48$ with one shared code (similar with independent codes and without filters), all within ± 0.02 and with no trend in n. So the exactly tight root condition needs no extra excess.

7 A second application: Meet-LWE

Setting. In Meet-LWE [17] the secret $s \in \{-1, 0, 1\}^n$ of an LWE instance $As + e = b \bmod q$ has relative weight ω ($\omega n/2$ entries of each sign). May writes s as a sum of two ternary vectors in many ways, as in a representation tree with digit set $\{-1, 0, 1\}$, and joins lists on exact q-ary coordinates of As, except at the root, where Odlyzko's locality-sensitive hashing matches As_1 and $b - As_2$ up to the error e (one bit per coordinate). We use the Rep-1 tree of [17, §7], in which a ± 1 splits as $(\pm 1, 0)$ or $(0, \pm 1)$ and a 0 as $(0, 0)$, $(1, -1)$ or $(-1, 1)$, with May's accounting: the q-ary coordinates fixed at level j give $c_j \le \rho_j$ bits, with ρ_j the representation count, so the expected number of surviving representations is at least one at every node and no survival recursion is needed. Lists have size $\ell_j = H_j - c_j$, the root costs $2\ell_1 - 1$, the merge of two level-j lists costs $2\ell_j - (c_{j-1} - c_j)$, and the leaves $H_d/2$. As in May's Table 5 the exponents are asymptotic in q, with the guessing cost ignored. We add the filters of Section 3 to every merge, with the cost terms of Proposition 3, and optimise the split laws of all levels and the filters together, as in Section 5; our depth d is May's depth minus one.

Results. Table 5 gives the exponents for May's key weights. Every point was re-costed by an evaluator written independently of the optimiser, and the unfiltered points also by May's own formulas. Re-optimising those formulas gives the column "May, re-opt."; it equals our best unfiltered value at every ω.[1] The filters gain 0.013–0.023 for $\omega \ge 0.375$ and 0.0057 at $\omega = 0.3$. The gain grows with ω, as denser keys produce more pattern waste.

			unfiltered / filtered		best over $d \le 7$		
ω	May [17]	May, re-opt.	$d = 3$	$d = 4$	unfiltered (d)	filtered (d)	gain
0.3	0.298	0.2972	0.2972 / 0.2936	0.2975 / 0.2922	0.2972 (3)	0.2915 (6)	0.0057
0.375	0.323	0.3206	0.3208 / 0.3177	0.3206 / 0.3092	0.3206 (4)	0.3076 (6)	0.0131
0.441	0.340	0.3369	0.3393 / 0.3364	0.3369 / 0.3219	0.3369 (4)	0.3197 (6)	0.0172
0.5	0.356	0.3511	0.3548 / 0.3518	0.3511 / 0.3352	0.3511 (4)	0.3343 (5)	0.0168
0.62	0.389	0.3877	0.3878 / 0.3830	0.3877 / 0.3708	0.3877 (4)	0.3698 (7)	0.0179
0.667	0.407	0.4068	0.4068 / 0.3970	0.4068 / 0.3843	0.4068 (3)	0.3843 (6)	0.0226

Table 5: Meet-LWE (Rep-1) time exponents $\log_2(T)/n$, asymptotic, guessing ignored. "May" is [17, Table 5]; "May, re-opt." is his Rep-1 formulas re-optimised over his depths 4–6. The columns $d = 3$ and $d = 4$ give our best points found at that depth, unfiltered / filtered; the last columns give the best found over $d = 3, \ldots, 7$, with the smallest depth attaining it. The gain is unfiltered minus filtered, both best over depths.

Where the waste is. May's "Match-and-Filter" checks the ternary split pattern after matching, so every matched pair with the wrong pattern is generated and discarded. At May's own parameters the filters gain nothing: at $\omega = 0.5$ and depth 3 the binding merges are those of the level-2 and level-3 lists, with little pattern waste ($p_2 \approx -0.06$, $p_3 \approx -0.02$; only the root merge, which does not bind, has $p_1 \approx -0.13$), and the leaves and the level-3 lists bind as well, which filters do not reduce; with the split laws fixed, the best filters give 0.3548, the same as without them (0.3548). Once the split laws are re-optimised together with the filters, the optimum moves to denser levels

[1] At $\omega \approx 0.44$–0.5 this is a small gain from one more level (for example $0.3548 \to 0.3511$ at $\omega = 0.5$, from May's depth 4 to 5); [17, §7] reports none, presumably for the concrete instances or the optimiser used there. We quote filter gains against our own best unfiltered values.

with more cancellations $0 \to (1,-1)$, where most matched pairs have the wrong pattern, and the binding merge of the level-2 lists has $p_2 \approx -0.14$ ($\omega = 0.5$, depth 5); the filters remove that waste. This is the mechanism of Section 5.

Depth. Filters gain only 0.003–0.010 at depth 3. The gain appears at the first extra level they make affordable, depth 4 in our convention (May's depth 5): at $\omega = 0.5$, going from depth 3 to 4 gains 0.0166 with filters against 0.0037 without, and the other weights show the same pattern (Table 5). Deeper trees add at most about 0.002. What the extra level contains is the mechanism above: denser, more wasteful levels that only pay once they are filtered. So filters make depth pay in Meet-LWE as in Subset Sum. Unlike the BBSS accounting of Section 5, May's accounting does not let a trivial extra level copy a shallower tree exactly, so the exponent need not be monotone in depth (for example 0.2972 at depth 3 against 0.2975 at depth 4 for $\omega = 0.3$, unfiltered); we report the best found at each depth and the best over depths.

Soundness. Lemma 6 carries over: coordinate permutations act on the columns of A and on s, the error e is untouched, and exact q-ary matching, the split checks and filter entry are all equivariant. A small-n simulation of a filtered merge shaped like the binding level-2 merge at $\omega = 0.5$ (parents of type $(4,4)$, children of type $(3,3)$ with one cancellation of each sign, $n = 20$, $q = 67$, two coordinates, complete cosets, `ntru_hybrid/bias_sim_meetlwe2.py`) gives planted survival 0.649 ± 0.011 without filters against $\sigma = 0.652$ (2000 draws), and 0.505 ± 0.011 with filters against $\sigma = 0.487$ (2000 draws); the same filtered merge with no planted parent gives $\mathbb{E}|O|/\mathbb{E}|V_t| = 0.487 \pm 0.001$ (1600 draws), so the filtered arm is within 2 standard errors of the lemma (+1.6), in the favourable direction.

Scope. These are asymptotic exponents in May's model. Concrete instances, with integer numbers of q-ary coordinates per level and the guessing cost, are future work.

8 Conclusion and open problems

Filters and depth attack the same waste from two sides, and together they give 0.2690; with depth the exponent converges to about 0.269. What remains is merge cost, so going substantially lower needs a structural change, such as a non-binary root with a fast join, non-modular information, or a top level that is not enumerated, rather than more depth. Open: quantum (walk or Grover) versions of filtered merges, as in [6]; non-binary roots and non-enumerative top levels; wider digit sets, which add at most about 0.0003 at fixed depth; and transferring the observation to decoding trees and to other ternary-LWE trees, where merges also generate pairs with the wrong split pattern (for Meet-LWE, see Section 7).

For designers of representation algorithms the takeaway is a simple test: at each merge, compute $p_j = H(\sigma_j) - 2H_j$. Dubiner filters can pay only where $2^{p_j n} \ll 1$, and do pay where such a merge binds, and then they free the depth that the waste otherwise prevents; where $p_j \approx 0$ they cannot help, and where the wasteful merge does not bind (the HGJ top merge, the sparse-key HKLS top merge) they do not. Lattice-based hybrids, such as guessing combined with batch-CVP, may be competitive at density one; whether they can be combined with filtered representation trees is open.

A Parameters and reproduction

In the numerical model the filter law π is a free parameter; at one optimum it drifted to $\pi_N \approx 0$, giving formal code rates of $34n$ and $128n$ at two levels, while all costs were unchanged on resetting

$\pi = Q_\Lambda$ (Remark 4). The rates in the tables are at $\pi = Q_\Lambda$.

Table 6 gives the depth-5 point (columns as in Table 3); the best points at depths 6–8 are its trivial extensions. The points are stored as

`floor/runs/polish/gen_m4_4_f_cloud.npy` (depth 4)
`floor/runs/polish/gen_m4_5_f_cloud.npy` (depth 5)
`floor/runs/polish/gen_m4_6_f_cloud.npy`, …, `_8_f_cloud.npy` (depths 6–8)

and were checked by an independent re-costing (`floor/recost.py`) and against the survival recursion S2 (`floor/finalize_cloud.py`). The Meet-LWE points are in `ntru_hybrid/points/`, generated into the paper by `ntru_hybrid/make_ml_numbers.py`. All numbers in this paper are generated by `floor/make_numbers.py` and `ntru_hybrid/make_ml_numbers.py`; file names are relative to the repository (see Data and code availability); the model is `gen_lsf.py`, the optimiser `gen_opt.py`, and the survival-recursion accounting `floor/recursive_floor.py` (variants S1/S2).

j	$\delta_j(-1)$	$\delta_j(0)$	$\delta_j(1)$	$\delta_j(2)$	ρ_j	c_j	s_j	ℓ_j
1	0.0505	0.6567	0.2851	0.0077	1.0148	0.8939	-0.0605	0.2316
2	0.0627	0.7527	0.1817	0.0030	0.7678	0.8283	0.0000	0.2027
3	0.0723	0.7950	0.1305	0.0022	0.7145	0.7145	0.0000	0.2254
4	0.0556	0.8583	0.0853	0.0008	0.4712	0.4712	0.0000	0.2608
5	0.0316	0.9212	0.0472	0.0000	0.2109	0.2109	0.0000	0.2636

merge of level j	pairs	p_j	$I(XY;\Lambda)$	entries gap	saving	passing
1	0.3571	-0.3571	0.1629	0.0373	-0.0882	0.2690
2	0.3397	-0.1081	0.2033	0.0663	-0.0707	0.2690
3	0.3370	-0.1344	0.1552	0.0435	-0.0681	0.2690
4	0.2783	-0.0529	0.0256	0.0081	-0.0093	0.2690
5	0.2668	-0.0060	0.0014	0.0008	+0.0002	0.2670

leaves $H_5/2 = 0.2372$; root excess $\rho_1 - c_1 = +0.1209$, $2s_1 = -0.1209$; memory 0.2690; $T = 0.26896$

Table 6: The depth-5 point, the best computed.

Data and code availability

- *Code:* available from the author on request, and to be released in a public repository with the final version. It contains the tree model and optimiser (`gen_lsf.py`, `gen_opt.py`, `cloud/polish_worker.py`), the independent evaluators (`floor/recost.py`, `ntru_hybrid/ml_recost.py`, `ntru_hybrid/may_rep1.py`), the survival-recursion check (`floor/recursive_floor.py`, `floor/finalize_cloud.py`), the bias, survival and sibling simulations, and the generators `floor/make_numbers.py` and `ntru_hybrid/make_ml_numbers.py`.
- *Data:* all reported points (`floor/runs/polish/`, `ntru_hybrid/points/`) and the logs of the checks are in the repository; the full logs of the cloud optimisation runs (about 21 MB) are available on request.

`floor/make_numbers.py` and `ntru_hybrid/make_ml_numbers.py` regenerate every number and table in this paper from the stored points; `floor/recost.py` and `ntru_hybrid/ml_recost.py` re-cost each point independently of the optimiser.

Acknowledgements

AI tools (Anthropic's Claude) were used for literature search, checking algebra, writing and running the numerical optimisations and simulations, independent re-costing of the reported optima, and drafting text. All results and text were checked by the author, who takes full responsibility for the content.

References

[1] E. Böhme, *Verbesserte Subset-Sum Algorithmen*, Master's thesis, Ruhr-Universität Bochum, 2011.

[2] A. Becker, L. Ducas, N. Gama and T. Laarhoven, New directions in nearest neighbor searching with applications to lattice sieving, in *Proc. SODA 2016*, SIAM, pp. 10–24.

[3] L. Both and A. May, Optimizing BJMM with nearest neighbors: full decoding in $2^{2n/21}$ and McEliece security, *International Workshop on Coding and Cryptography (WCC 2017)*, 2017.

[4] L. Both and A. May, Decoding linear codes with high error rate and its impact for LPN security, in *PQCrypto 2018*, LNCS, Springer, pp. 25–46. IACR ePrint 2017/1139.

[5] A. Becker, J.-S. Coron and A. Joux, Improved generic algorithms for hard knapsacks, in *EUROCRYPT 2011*, LNCS 6632, Springer, pp. 364–385.

[6] X. Bonnetain, R. Bricout, A. Schrottenloher and Y. Shen, Improved classical and quantum algorithms for subset-sum, in *ASIACRYPT 2020*, LNCS, Springer. IACR ePrint 2020/168, arXiv:2002.05276.

[7] T. Christiani, A framework for similarity search with space-time tradeoffs using locality-sensitive filtering, in *Proc. SODA 2017*, SIAM, pp. 31–46.

[8] I. Csiszár and J. Körner, *Information Theory: Coding Theorems for Discrete Memoryless Systems*, 2nd ed., Cambridge University Press, 2011.

[9] M. Dubiner, Bucketing coding and information theory for the statistical high-dimensional nearest-neighbor problem, *IEEE Trans. Inf. Theory* **56** (2010), no. 8, 4166–4179.

[10] A. Esser and A. May, Better sample – random subset sum in $2^{0.255n}$ and its impact on decoding random linear codes, arXiv:1907.04295 (withdrawn).

[11] E. Horowitz and S. Sahni, Computing partitions with applications to the knapsack problem, *J. ACM* **21** (1974), 277–292.

[12] N. Howgrave-Graham and A. Joux, New generic algorithms for hard knapsacks, in *EUROCRYPT 2010*, LNCS 6110, Springer, pp. 235–256.

[13] W. Hoeffding, Probability inequalities for sums of bounded random variables, *J. Amer. Statist. Assoc.* **58** (1963), no. 301, 13–30.

[14] M. Hhan, J. Kim, C. Lee and Y. Son, A hybrid of lattice-reduction and Meet-LWE via near-collision on Babai's plane, *Des. Codes Cryptogr.* **94** (2026), no. 4, art. 83. IACR ePrint 2022/1473.

[15] E. Kirshanova and A. May, How to find ternary LWE keys using locality sensitive hashing, in *IMACC 2021*, LNCS, Springer. IACR ePrint 2021/1255.

[16] Y. Li, From subset-sum to decoding: improved classical and quantum algorithms via ternary representation technique, *Information* **16** (2025), no. 10, 887.

[17] A. May, How to meet ternary LWE keys, in *CRYPTO 2021*, LNCS 12826, Springer, pp. 701–731. IACR ePrint 2021/216.

[18] A. May and I. Ozerov, On computing nearest neighbors with applications to decoding of binary linear codes, in *EUROCRYPT 2015*, LNCS 9056, Springer, pp. 203–228.

[19] I. Ozerov, *Combinatorial Algorithms for Subset Sum Problems*, PhD thesis, Ruhr-Universität Bochum, 2016.

[20] R. Schroeppel and A. Shamir, A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems, *SIAM J. Comput.* **10** (1981), 456–464.